Corrigés : Web Sémantique, RDF et RDFS

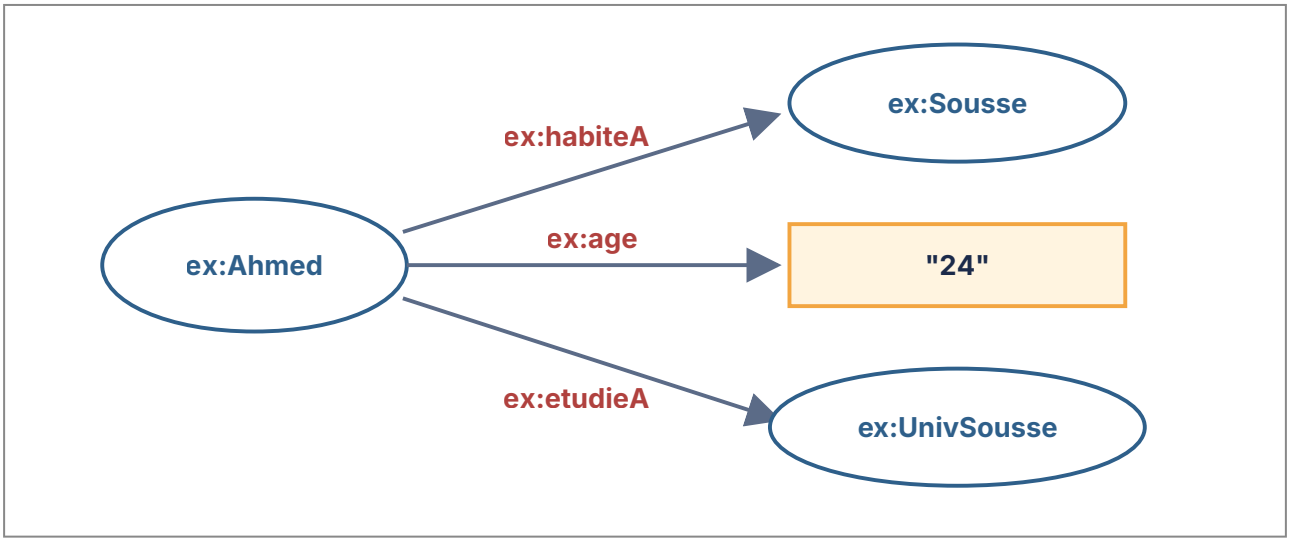
Corrigé 1 : Sujet, prédicat, objet

	Sujet	Prédicat	Objet	Type de l'objet
1	la page accueil.html	auteur	"Salma Ben Ali"	littéral (texte)
2	Tunis	estCapitaleDe	Tunisie	ressource
3	Les Misérables	datePublication	"1862"	littéral (valeur)

À retenir : *le sujet est toujours une ressource (identifiée par une URI). L'objet est soit une autre ressource (on peut alors « continuer » le graphe), soit un littéral (le graphe s'arrête là). C'est la même idée que dans le cours : (ressource, propriété, valeur).*

Corrigé 2 : Décomposer une phrase en triplets

Sujet	Prédicat	Objet
ex:Ahmed	ex:habiteA	ex:Sousse (ressource)
ex:Ahmed	ex:age	"24" (littéral)
ex:Ahmed	ex:etudieA	ex:UnivSousse (ressource)



À retenir : *un graphe RDF est simplement l'ensemble de ces triplets dessinés ensemble. Les trois flèches partent du même nœud ex:Ahmed.*

Corrigé 3 : URI, URN, tag ou littéral ?

	Réponse
1. "Salma"	littéral (texte)
2. http://exemple.org/personne/salma	URI de type **URL**
3. 1862	littéral (nombre)
4. urn:ietf:rfc:2141	URI de type **URN**
5. tag:websemantique.org,…	URI de type **tag URI**
6. "application/pdf"	littéral (c'est la valeur de *dc:format* dans l'exemple du cours)

Bonus : non. Un littéral est une valeur, il ne peut apparaître qu'en position d'**objet**. Le sujet est toujours une ressource.

À retenir : *l'URI sert à identifier de façon unique une ressource. Grâce à elle, plusieurs personnes peuvent ajouter des métadonnées sur la même ressource et on peut les agréger.*

Corrigé 4 : Écrire un triplet en RDF/XML

```
<?xml version="1.0" encoding="UTF-8"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
  <rdf:Description rdf:about="http://grandshommes.org/Platon">
    <rdf:type rdf:resource="http://description.org/schema/personnes/Philosophe"/>
  </rdf:Description>
</rdf:RDF>
```

- rdf:about : la ressource **décrite** (le sujet).
- rdf:type : la propriété (le prédicat).
- rdf:resource : l'objet, ici une autre ressource (et pas un texte).

Remarque : *sur les slides du cours, ces attributs sont écrits about et resource sans préfixe. La forme actuelle et correcte est rdf:about et rdf:resource, que j'utilise ici.*

Corrigé 5 : Lire un exemple Dublin Core

1. **4 triplets** : un par propriété (creator, title, date, format), tous avec le même sujet.
2. (http://exemple.org/docs/rapport-2024.pdf, dc:creator, "Salma Ben Ali")
3. Ce sont les **espaces de noms** : ils associent un préfixe (*rdf:*, *dc:*) à un vocabulaire identifié par une URI. *dc:creator* veut donc dire « la propriété creator du Dublin Core ».
4. Le préfixe dc n'est plus déclaré : le fichier n'est plus un XML valide et un analyseur le rejette.

À retenir : *les espaces de noms rendent le vocabulaire extensible : on peut mélanger plusieurs vocabulaires dans un même document sans conflit de noms.*

Corrigé 6 : Écrire une description Dublin Core

```
<?xml version="1.0" encoding="UTF-8"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:dc="http://purl.org/dc/elements/1.1/">
  <rdf:Description rdf:about="http://exemple.org/livres/miserables">
    <dc:title>Les Misérables</dc:title>
    <dc:creator>Victor Hugo</dc:creator>
    <dc:date>1862</dc:date>
    <dc:type>Text</dc:type>
  </rdf:Description>
</rdf:RDF>
```

Vérification : *4 propriétés = 4 triplets, un seul rdf:Description, deux espaces de noms déclarés, toutes les balises ouvertes sont refermées.*

Corrigé 7 : Trouver les erreurs

1. Le préfixe dc: est utilisé mais son espace de noms n'est **pas déclaré** (il manque xmlns:dc).
2. La balise <dc:creator> n'est **jamais refermée**.
3. La balise de fermeture est dans l'**ordre incorrect** : une fois dc:creator refermée, on ferme rdf:Description, puis rdf:RDF (en XML, les balises doivent être bien imbriquées).

Selon la façon de compter, la 3e erreur est la conséquence de la 2e : une balise non fermée casse l'imbrication de tout ce qui suit.

```
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:dc="http://purl.org/dc/elements/1.1/">
  <rdf:Description rdf:about="http://exemple.org/livre/1">
    <dc:title>Candide</dc:title>
    <dc:creator>Voltaire</dc:creator>
  </rdf:Description>
</rdf:RDF>
```

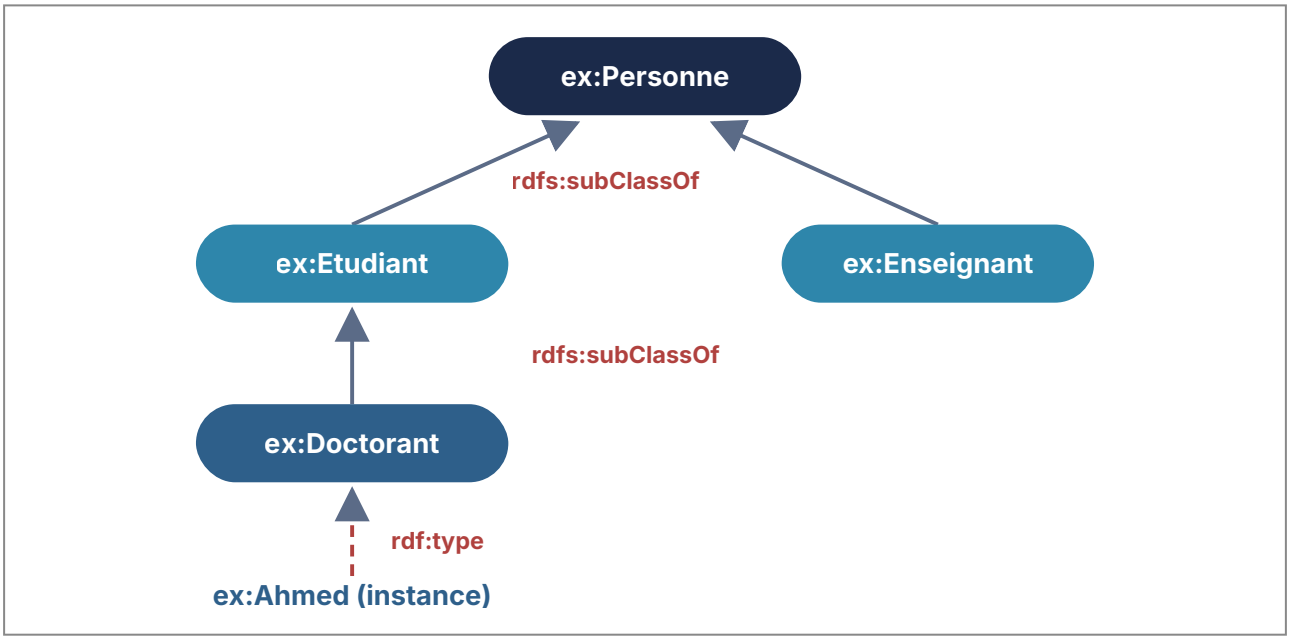
À retenir : *un fichier RDF/XML est d'abord un fichier XML : il doit être bien formé avant d'être du RDF correct.*

Corrigé 8 : Classes et instances

1. **Classes** : ex:Personne, ex:Etudiant, ex:Cours. **Instances** : ex:Salma (de Etudiant) et ex:WebSemantique (de Cours).
2. Le triplet **n°2** : *ex:Etudiant rdfs:subClassOf ex:Personne*.
3. Il n'est **pas écrit**, mais on peut le **déduire** : Salma est un Etudiant (triplet 3) et tout Etudiant est une Personne (triplet 2). Le nouveau fait est *ex:Salma rdf:type ex:Personne*.

À retenir : *rdf:type relie une instance à sa classe ; rdfs:subClassOf relie deux classes. Un moteur d'inférence combine les deux.*

Corrigé 9 : Hiérarchie de classes



1. ex:Ahmed est instance de **ex:Doctorant** (écrit), de **ex:Etudiant** et de **ex:Personne** (déduits en remontant la hiérarchie).
2. **Non.** Les deux classes ont le même parent (Personne) mais aucun lien de sous-classe n'est déclaré entre elles : ce sont des classes « sœurs ».

À retenir : *l'héritage RDFS est transitif : si $A \subset B$ et $B \subset C$ alors $A \subset C$. Une instance de A est donc aussi instance de B et de C.*

Corrigé 10 : Domaine et portée (domain / range)

```
<?xml version="1.0" encoding="UTF-8"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#">

  <rdfs:Class rdf:about="http://exemple.org/Personne"/>

  <rdfs:Class rdf:about="http://exemple.org/Enseignant">
    <rdfs:subClassOf rdf:resource="http://exemple.org/Personne"/>
  </rdfs:Class>

  <rdfs:Class rdf:about="http://exemple.org/Cours"/>

  <rdf:Property rdf:about="http://exemple.org/enseigne">
    <rdfs:domain rdf:resource="http://exemple.org/Enseignant"/>
    <rdfs:range  rdf:resource="http://exemple.org/Cours"/>
  </rdf:Property>
</rdf:RDF>
```

À retenir : *la propriété est déclarée de type rdf:Property ; rdfs:domain désigne le sujet attendu (Enseignant) et rdfs:range l'objet attendu (Cours).*

Corrigé 11 : Inférence avec domain et range

1. Trois déductions :
    - *ex:Karim rdf:type ex:Enseignant* (grâce au **domain**)
    - *ex:BasesDeDonnees rdf:type ex:Cours* (grâce au **range**)
    - *ex:Karim rdf:type ex:Personne* (Enseignant est une sous-classe de Personne)
2. RDFS déduit : *ex:Sami rdf:type ex:Enseignant* (domain) et **ex:Karim rdf:type ex:Cours** (range). Il ne signale **aucune erreur** : Karim devient simplement « un cours » en plus d'être « un enseignant ».

Point important : *en RDFS, domain et range ne sont **pas des contraintes** qui interdisent une donnée, ce sont des **règles d'inférence** qui ajoutent des types. Pour détecter l'incohérence « une personne ne peut pas être un cours », il faut un langage plus expressif comme **OWL** (par exemple avec owl:disjointWith).*

Corrigé 12 : Mini-projet : le schéma d'une bibliothèque

```
@prefix rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix ex:   <http://exemple.org/> .

# --- Schéma ---
ex:Livre    a rdfs:Class .
ex:Roman    a rdfs:Class ; rdfs:subClassOf ex:Livre .
ex:Personne a rdfs:Class .
ex:Auteur   a rdfs:Class ; rdfs:subClassOf ex:Personne .

ex:ecritPar a rdf:Property ;
    rdfs:domain ex:Livre ;
    rdfs:range  ex:Auteur .

ex:titre a rdf:Property ;
    rdfs:domain ex:Livre ;
    rdfs:range  rdfs:Literal .

# --- Données ---
ex:Candide a ex:Roman ;
    ex:titre    "Candide" ;
    ex:ecritPar ex:Voltaire .
```

Déductions RDFS :

- *ex:Candide rdf:type ex:Livre* (Roman est une sous-classe de Livre)
- *ex:Voltaire rdf:type ex:Auteur* (**range** de ecritPar)
- *ex:Voltaire rdf:type ex:Personne* (Auteur est une sous-classe de Personne)

À retenir : *dans Turtle, a est un raccourci pour rdf:type, le point-virgule ; répète le même sujet, et le point . termine le bloc. On n'a écrit que 3 faits sur Voltaire et Candide, RDFS en a déduit 3 de plus.*

Corrigé du QCM

1 → b) Un sujet, un prédicat et un objet

Sujet, prédicat, objet : la base de RDF.

2 → a) Une ressource (URI)

Le sujet est toujours une ressource identifiée par une URI.

3 → b) Dire qu'une ressource est instance d'une classe

rdf:type relie une instance à sa classe.

4 → a) Salma est une Personne

Un moteur d'inférence remonte la hiérarchie : Salma est aussi une Personne.

5 → a) Le sujet de la propriété

Le domain donne la classe du sujet.

6 → b) L'objet de la propriété

Le range donne la classe de l'objet.

7 → c) Non, il faut OWL

RDFS n'a pas de classes disjointes : il faut OWL (owl:disjointWith).

8 → b) Il faut une entente sur le vocabulaire

Le cours cite la nécessité d'une entente sur le vocabulaire.

Niveau 1 : débutant

Class, Property ou Description ?

Corrigé 13

#	Élément	Réponse	Pourquoi
a	`Voiture`	`rdfs:Class`	Un type (ensemble de voitures)
b	`possède`	`rdf:Property`	Une relation (verbe)
c	`ex:MaVoiture`	`rdf:Description`	Un individu précis, avec un URI
d	`dateNaissance`	`rdf:Property`	Une propriété qui prend une valeur (range = Literal)
e	`Ville`	`rdfs:Class`	Un type
f	`Tunis`	`rdf:Description`	Une ville précise, pas un type
g	`habiteEn`	`rdf:Property`	Une relation (domain = Personne, range = Ville)
h	`Mohamed`	`rdf:Description`	Une personne précise

Astuce : *« nom de type » → `rdfs:Class` ; « verbe ou propriété » → `rdf:Property` ; « individu précis » → `rdf:Description`.*

Corrigé 14

(a) Le schéma, écrit une seule fois :

```
<rdfs:Class rdf:ID="Personne"/>
<rdfs:Class rdf:ID="Ville"/>

<rdf:Property rdf:ID="habiteEn">
  <rdfs:domain rdf:resource="#Personne"/>
  <rdfs:range  rdf:resource="#Ville"/>
</rdf:Property>
```

Corrigé 14 (b)

(b) Les données, pour chaque individu :

```
<rdf:Description rdf:about="http://exemple.tn/Mohamed">
  <rdf:type rdf:resource="#Personne"/>
  <ex:habiteEn rdf:resource="http://exemple.tn/Tunis"/>
</rdf:Description>

<rdf:Description rdf:about="http://exemple.tn/Tunis">
  <rdf:type rdf:resource="#Ville"/>
</rdf:Description>
```

Remarque : même sans `rdf:type`, RDFS le déduirait depuis le domain et le range. On l'écrit pour la clarté.

Corrigé 15

Erreur : on a utilisé `rdf:Description` au lieu de `rdfs:Class`. La ligne dit « il existe une chose nommée Livre », pas « Livre est une classe ».

Solution 1 (la plus courte) :

```
<rdfs:Class rdf:ID="Livre"/>
```

Solution 2 (même sens) : garder Description et ajouter `rdf:type` :

```
<rdf:Description rdf:ID="Livre">
  <rdf:type rdf:resource="http://www.w3.org/2000/01/rdf-schema#Class"/>
</rdf:Description>
```

Niveau 2

Corrigé 16

4 déductions (une en double) : **Sara** est une Personne ; **Minou** est un Animal et un Mammifère (Animal apparaît deux fois : par le range et par subClassOf).

```
Sara  rdf:type  Personne     (domain de possède)
Minou rdf:type  Animal       (range de possède)
Minou rdf:type  Mammifère    (Chat ⊂ Mammifère)
Minou rdf:type  Animal       (Mammifère ⊂ Animal, déjà obtenu)
```

Règle : le `domain` concerne le sujet, le `range` concerne l'objet, et `subClassOf` est transitif.

Corrigé 17

On déduit : `Leila estParentDe Omar` (sous-propriété), puis `Leila rdf:type Personne` et `Omar rdf:type Personne`.

Oui : tout triplet `estMèreDe` est aussi un triplet `estParentDe` ; le domain et le range de la propriété parente s'appliquent donc à la fille.

Corrigé 18

Erreur 1 : le `range` doit être une ressource (`rdf:resource`), pas le texte « entier ». On utilise `xsd:integer`.

Erreur 2 : `subClassOf` est inversé. Dans la définition d'Enseignant : `subClassOf Personne` (du particulier au général). Le code d'origine disait « toute Personne est un Enseignant ».

Erreur 3 : `rdf:type` doit pointer vers une classe (`#Enseignant`), pas vers une propriété (`#enseigne`).

Code corrigé :

```
<rdfs:range rdf:resource="http://www.w3.org/2001/XMLSchema#integer"/>

<rdfs:Class rdf:ID="Enseignant">
  <rdfs:subClassOf rdf:resource="#Personne"/>
</rdfs:Class>

<rdf:type rdf:resource="#Enseignant"/>
```

Corrigé 19

(a) Le schéma :

```
<rdfs:Class rdf:ID="Personne"/>
<rdfs:Class rdf:ID="Auteur">
  <rdfs:subClassOf rdf:resource="#Personne"/>
</rdfs:Class>
<rdfs:Class rdf:ID="Livre"/>

<rdf:Property rdf:ID="titre">
  <rdfs:domain rdf:resource="#Livre"/>
  <rdfs:range  rdf:resource="http://www.w3.org/2000/01/rdf-schema#Literal"/>
</rdf:Property>
<rdf:Property rdf:ID="ecritPar">
  <rdfs:domain rdf:resource="#Livre"/>
  <rdfs:range  rdf:resource="#Auteur"/>
</rdf:Property>
```

Corrigé 19 (b)

Les données : `titre` est un littéral (texte) ; `ecritPar` utilise `rdf:resource` (ressource).

```
<rdf:Description rdf:about="http://exemple.tn/LesMiserables">
  <rdf:type rdf:resource="#Livre"/>
  <ex:titre>Les Misérables</ex:titre>
  <ex:ecritPar rdf:resource="http://exemple.tn/VictorHugo"/>
</rdf:Description>

<rdf:Description rdf:about="http://exemple.tn/VictorHugo">
  <rdf:type rdf:resource="#Auteur"/>
</rdf:Description>
```

Niveau 3

Corrigé 20

(a) Non, pas de contradiction. RDFS n'interdit rien.

(b) On déduit `Sami rdf:type Enseignant` (domain). Sami est donc à la fois Étudiant et Enseignant.

(c) Il faut **OWL** : on déclare `owl:disjointWith` entre Étudiant et Enseignant ; le raisonneur détecte alors une incohérence.

```
<owl:Class rdf:about="#Etudiant">
  <owl:disjointWith rdf:resource="#Enseignant"/>
</owl:Class>
```

Corrigé 21

(a) On déduit `Ali rdf:type Homme` **et** `Ali rdf:type Femme`.

(b) Avec « **et** » (intersection) : chaque domain ajoute une déduction, sans faire de choix.

(c) Non, RDFS ne rejette pas. La conception est logiquement mauvaise ; pour l'empêcher, on utilise OWL (`disjointWith`) ou un domain unique plus général (Personne).

Corrigé 22

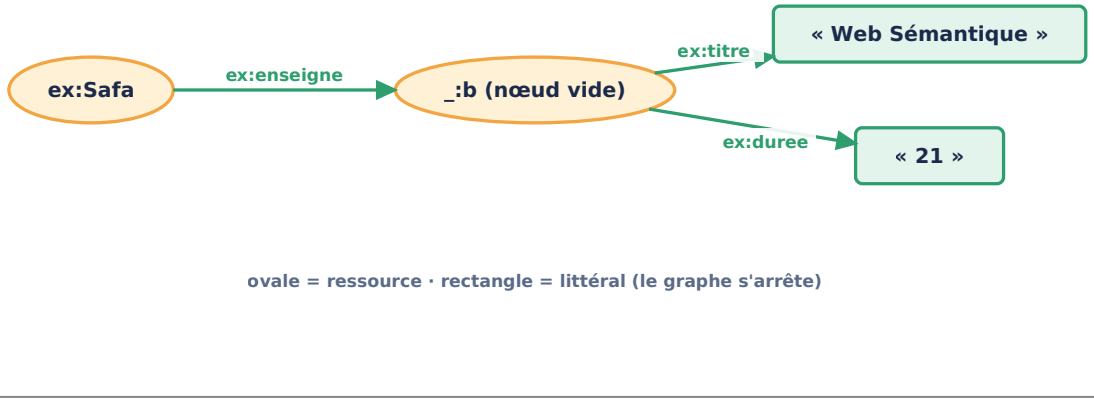
Réponse : **4 triplets déduits**.

```
Amine    rdf:type  Étudiant   (domain de inscritA)
Amine    rdf:type  Personne   (Étudiant ⊂ Personne)
Mastère  rdf:type  Formation  (range de inscritA)
Mastère  rdf:type  Programme  (Formation ⊂ Programme)
```

Corrigé 23

(a) 3 triplets :

```
1) ex:Safa  ex:enseigne  _:b
2) _:b      ex:titre     "Web Sémantique"
3) _:b      ex:duree     "21"
```



(c) Le nœud intermédiaire est un **nœud vide (blank node)** : une ressource sans URI. Ressources : Safa et le nœud vide. Littéraux : « Web Sémantique » et « 21 ».

Niveau 4

Corrigé 24
(a) Le schéma :

```
<rdfs:Class rdf:ID="Personne"/>
<rdfs:Class rdf:ID="Employe"><rdfs:subClassOf rdf:resource="#Personne"/></rdfs:Class>
<rdfs:Class rdf:ID="Manager"><rdfs:subClassOf rdf:resource="#Employe"/></rdfs:Class>
<rdfs:Class rdf:ID="Departement"/>

<rdf:Property rdf:ID="travailleDans">
  <rdfs:domain rdf:resource="#Employe"/>
  <rdfs:range  rdf:resource="#Departement"/>
</rdf:Property>
<rdf:Property rdf:ID="dirige">
  <rdfs:subPropertyOf rdf:resource="#travailleDans"/>
  <rdfs:domain rdf:resource="#Manager"/>
  <rdfs:range  rdf:resource="#Departement"/>
</rdf:Property>
```

Corrigé 24 (b)
Les 5 triplets déduits :

```
1) Nour    travailleDans  Finance      (subPropertyOf)
2) Nour    rdf:type  Manager       (domain de dirige)
3) Nour    rdf:type  Employé       (Manager ⊂ Employé)
4) Nour    rdf:type  Personne      (Employé ⊂ Personne)
5) Finance rdf:type  Département   (range de dirige)
```

Remarque : après avoir déduit `Nour travailleDans Finance`, le domain de `travailleDans` (Employé) donne le même résultat (Employé) : rien de nouveau.

Corrigé 25
(a) Oui : Rami est un Étudiant (domain) et Étudiant ⊂ Personne.

(b) Oui : Algo est un Cours (range) et Cours ⊂ UnitéEnseignement.

(c) On ne sait pas : aucune règle ne dit qu'il est Enseignant. On ne le déduit pas, mais on ne dit pas non plus que c'est faux.

(d) On ne sait pas : RDFS ne déduit jamais de négation. C'est l'hypothèse du **monde ouvert** : l'absence d'information n'est pas une négation. Pour nier, il faut OWL.

Corrigé 26
On déduit `Safa rdf:type Formateur`. Le cycle signifie que les deux classes sont **équivalentes** : elles ont **exactement les mêmes individus**. C'est l'équivalent de `owl:equivalentClass` en OWL.

Corrigé 27

(a) Le schéma :

```
<rdfs:Class rdf:ID="Personne"/>
<rdfs:Class rdf:ID="Utilisateur"><rdfs:subClassOf rdf:resource="#Personne"/></rdfs:Class>
<rdfs:Class rdf:ID="Publication"/>
<rdfs:Class rdf:ID="Commentaire"><rdfs:subClassOf rdf:resource="#Publication"/></
rdfs:Class>

<rdf:Property rdf:ID="publie">
  <rdfs:domain rdf:resource="#Utilisateur"/>
  <rdfs:range  rdf:resource="#Publication"/>
</rdf:Property>
<rdf:Property rdf:ID="commente">
  <rdfs:subPropertyOf rdf:resource="#publie"/>
  <rdfs:domain rdf:resource="#Utilisateur"/>
  <rdfs:range  rdf:resource="#Commentaire"/>
</rdf:Property>
<rdf:Property rdf:ID="pseudo">
  <rdfs:domain rdf:resource="#Utilisateur"/>
  <rdfs:range  rdf:resource="http://www.w3.org/2000/01/rdf-schema#Literal"/>
</rdf:Property>
```

Corrigé 27 (b)

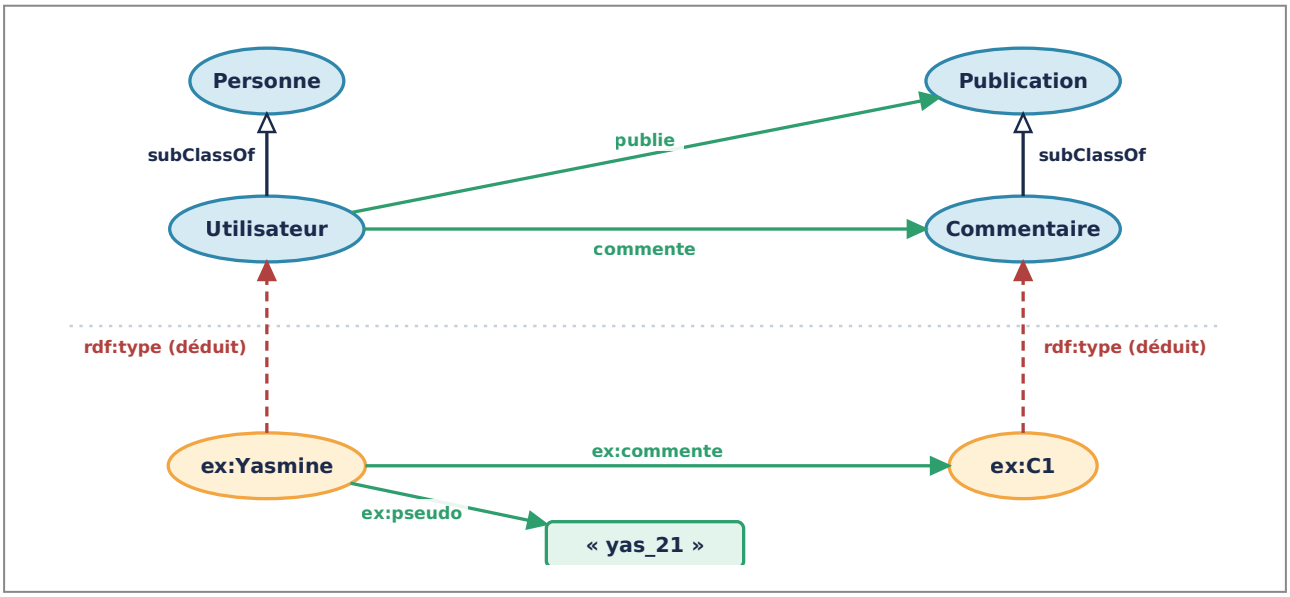
Les données :

```
<rdf:Description rdf:about="http://exemple.tn/Yasmine">
  <ex:commente rdf:resource="http://exemple.tn/C1"/>
  <ex:pseudo>yas_21</ex:pseudo>
</rdf:Description>
```

Corrigé 27 (c)

Les 5 triplets déduits :

```
1) Yasmine  rdf:type  Utilisateur   (domain)
2) Yasmine  rdf:type  Personne      (Utilisateur ⊂ Personne)
3) C1       rdf:type  Commentaire   (range de commente)
4) C1       rdf:type  Publication   (Commentaire ⊂ Publication)
5) Yasmine  ex:publie C1            (subPropertyOf)
```



On voit que `commente` produit deux effets : par son range on sait que C1 est un Commentaire, et par `subPropertyOf` on déduit aussi `publie`.